

Eventos en tiempo real

Pasar de una base de datos que hace todo, a un pipeline donde cada componente hace una cosa — y la señal llega a la pantalla del operador sin que nadie tenga que preguntar por ella. Con el mismo criterio para la multimedia que acompaña al evento.

Latencia subsegundo

Escala sin cargar la base

Portabilidad de motor

Cero pérdida de señales confirmadas

Multimedia unificada

Documento de discusión Análisis técnico detallado por separado

SQL Server no es la base de datos. Es la arquitectura.

El motor relacional acumuló ocho responsabilidades que en cualquier otro sistema estarían separadas. No es un problema de rendimiento de la base: es que todo compite por el mismo recurso.

Cuando la recepción de señales, la ejecución de reglas, la coordinación entre operadores y la consulta de las pantallas ocurren dentro del mismo proceso, no hay forma de escalar una sin afectar a las demás. Tampoco de reemplazar una sin tocar el resto.

API de ingreso

Motor de reglas

Cola de trabajo

Coordinador de concurrencia

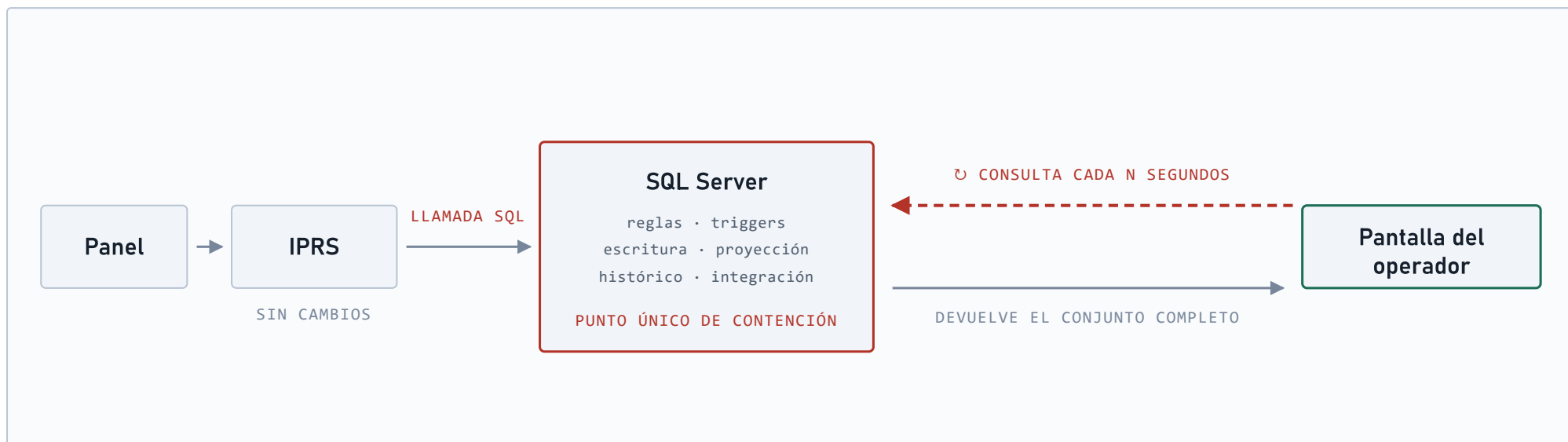
Almacenamiento operativo

Proyección de pantallas

Bitácora e histórico

El dato avanza. La pantalla va a buscarlo.

El panel se conecta a **IPRS**, que interpreta el protocolo y ejecuta un procedimiento en la base. Hasta ahí todo fluye en una dirección. El último tramo se invierte: nadie le avisa al operador, es su pantalla la que vuelve a preguntar cada tantos segundos, haya novedades o no.



EL INTERCAMBIO El intervalo es configurable, pero acortarlo multiplica las consultas contra la misma base que está recibiendo las señales. Bajar la latencia sube la carga: es un intercambio, no una mejora.

Tres señales idénticas, esperas distintas.

El mismo eje de tiempo para los dos modelos. Arriba, la pantalla consulta en ciclo; abajo, recibe. Lo que aparece no es sólo que hoy se tarda más — es que **cuánto se tarda depende de la suerte**: del momento exacto en que llegó la señal respecto del próximo ciclo.



SOBRE EL EJEMPLO Se grafica un ciclo de 3 s por legibilidad. Con ciclos más largos las esperas crecen en proporción: el argumento se refuerza.

Sumar operadores empeora el tiempo de respuesta de todos.

Consultar en lugar de recibir tiene tres consecuencias que se refuerzan entre sí. La tercera es la que convierte un problema técnico en un límite de negocio.

La latencia la fija el reloj, no el sistema

El tiempo de llegada a la pantalla no depende de qué tan rápido se procesó la señal, sino de cuánto falta para la próxima consulta.

La carga es constante

Cada pantalla abierta consulta al mismo ritmo haya cero eventos o mil. Se paga el costo de la vigilancia incluso cuando no pasa nada.

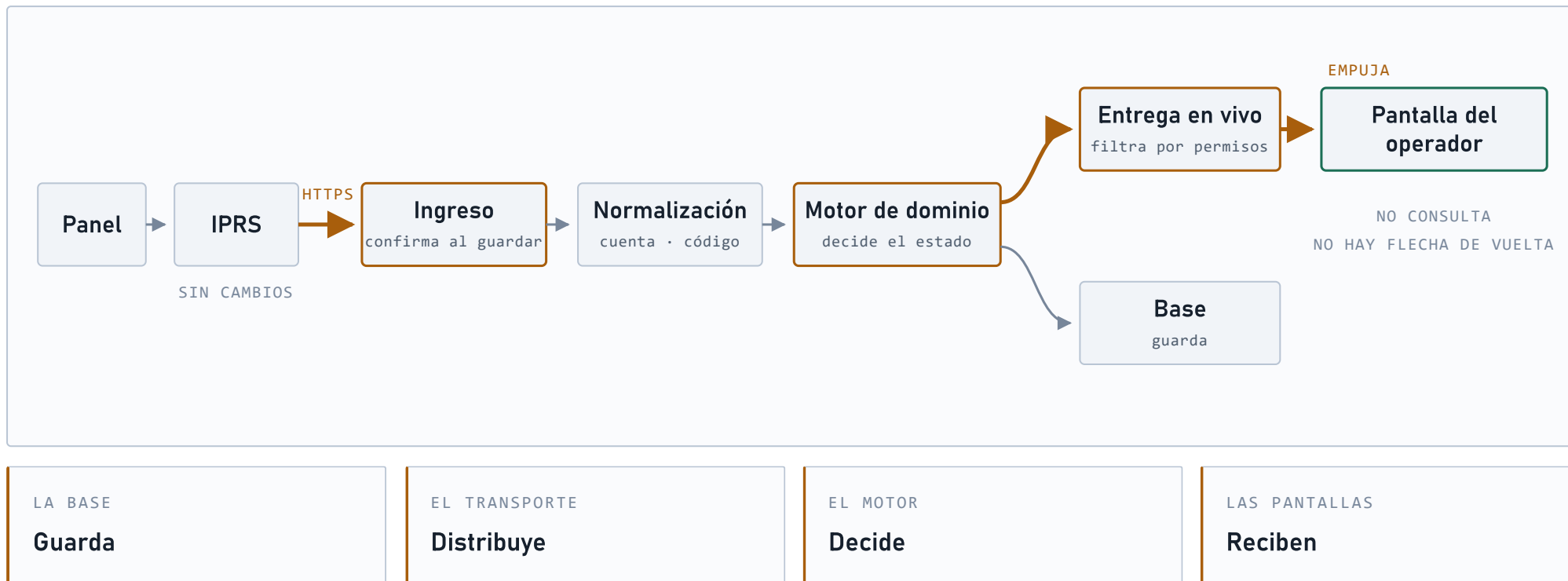
La escala trabaja en contra

Más operadores significa más consultas sobre el mismo recurso que además tiene que recibir y procesar las señales. Crecer degrada el servicio.

Es el punto donde la arquitectura empieza a decidir el negocio: la cantidad de puestos de monitoreo que una instalación puede sostener deja de ser una decisión comercial y pasa a ser un límite técnico.

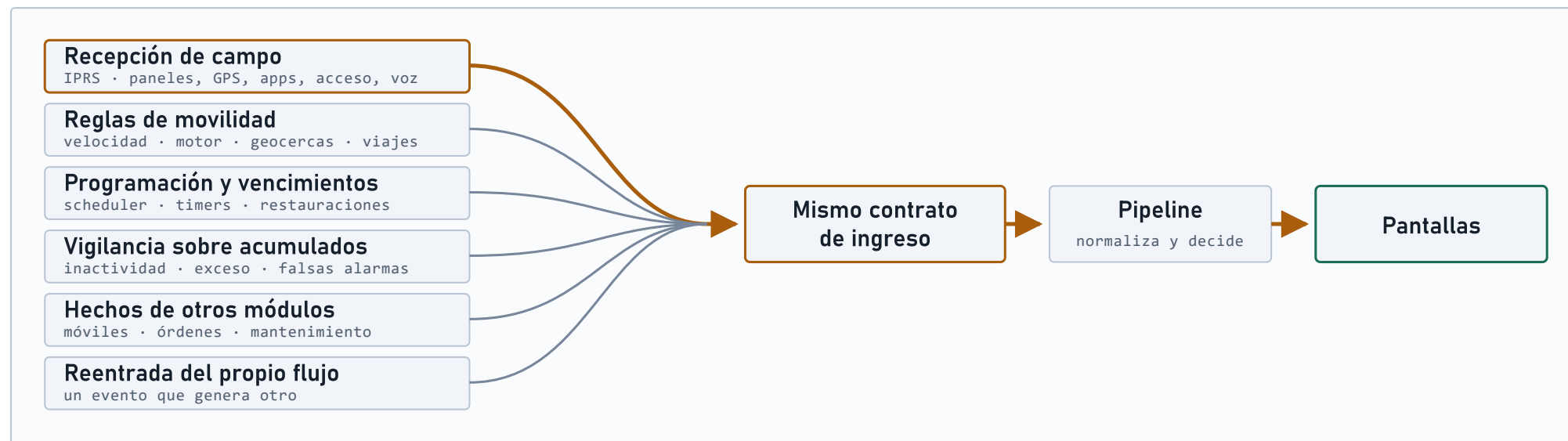
El panel sigue hablando con IPRS. IPRS deja de hablar con la base.

Mismo panel, mismo IPRS, mismos parsers de protocolo. Lo único que cambia es **por dónde sale IPRS**: en vez de ejecutar un procedimiento en la base, hace una llamada HTTP al servicio de ingreso.



IPRS es uno de seis generadores de eventos.

IPRS es el más visible porque es el que habla con los paneles, pero no es el único que crea eventos. Hoy los seis los generan **adentro de SQL**, y por eso los seis heredan el mismo modelo de consulta. En el diseño propuesto los seis entran por el mismo contrato.



POR QUÉ IMPORTA Define el alcance real del trabajo. Migrar IPRS no termina la migración: deja cinco generadores escribiendo en la base por el camino viejo. Que todos entren por el mismo contrato es lo que evita mantener dos formas de crear un evento.

Tiempo de respuesta

Hoy la latencia hasta la pantalla la define el intervalo de refresco. En el diseño propuesto la define el pipeline — y el pipeline se puede medir, optimizar y monitorear.

La diferencia de fondo no es el número. Es que pasa a ser un compromiso medible en lugar de un efecto lateral de una configuración. Se puede poner en un acuerdo de servicio, alertar cuando se degrada y saber en qué etapa se perdió el tiempo.

A VALIDAR Es un objetivo de diseño, no una medición. Confirmarlo con volúmenes y topología reales es parte del trabajo de cada módulo, no de una etapa previa.

< 1 s

OBJETIVO P95 · DESDE EL INGRESO CONFIRMADO HASTA LA PANTALLA DEL OPERADOR CONECTADO

HOY La latencia depende del intervalo de refresco. No se mide, se configura.

PROPUESTO La latencia depende del pipeline. Se instrumenta etapa por etapa.

Las pantallas dejan de ser carga de base de datos.

Es el cambio con mayor efecto sobre el costo de crecer. Hoy cada operador conectado es tráfico contra el mismo motor que recibe las señales; en el diseño propuesto, no.

HOY

Operadores y señales compiten

Cada pantalla abierta consulta la base en ciclo. Duplicar los puestos de monitoreo duplica esa carga, y lo hace justo sobre el recurso que también tiene que procesar la recepción.

PROPUESTO

Los operadores no tocan la base

La entrega en vivo lee el flujo una vez y lo distribuye en memoria a todas las conexiones. Sumar operadores agrega conexiones, que se escalan agregando instancias del componente de entrega — no ampliando la base.

La consecuencia comercial: la cantidad de puestos que una instalación soporta deja de estar atada al dimensionamiento del motor de base de datos. Crecer se vuelve un problema de agregar instancias, que es barato y previsible.

El motor de base deja de ser una decisión irreversible.

Hoy las reglas de negocio están escritas en el dialecto de un producto puntual. Eso significa que cambiar de motor no es migrar datos: es reescribir el negocio.

Al mover esas reglas a código versionado, la base vuelve a ser lo que debería ser — un lugar donde guardar cosas. Y el producto puede ofrecerse sobre el motor que cada cliente, cada regulación o cada modelo de costos requiera.

SE ABRE Instalaciones sobre motores sin licenciamiento propietario, donde hoy el costo de licencia condiciona la venta.

SE ABRE Despliegue en la nube o en servidores del cliente con el mismo producto, sin dos bases de código.

SE ABRE Reglas de negocio con pruebas automatizadas, revisión de cambios e historial — hoy imposible dentro de procedimientos almacenados.

HONESTO La portabilidad no es gratis ni automática. Exige una matriz explícita de motores soportados y pruebas de integración reales contra cada uno. Prometer «cualquier base» sin esa suite es prometer algo que no se verificó.

Nada confirmado se pierde.

En monitoreo de alarmas, perder una señal es peor que entregarla tarde. El diseño trata la durabilidad como un requisito explícito en cada frontera, no como una consecuencia de que todo esté en una transacción de base.

Si se corta el enlace

La señal queda en disco del lado del cliente y se reenvía cuando vuelve la conexión. La recepción sigue funcionando aunque la plataforma central no esté accesible.

Si se cae un proceso

El trabajo no confirmado se vuelve a entregar al reiniciar. Cada componente está diseñado para tolerar recibir el mismo mensaje dos veces sin duplicar la alarma.

Si se cae la base

La recepción no se detiene: el trabajo se acumula y se procesa al recuperarse, en lugar de rechazar señales mientras dura la indisponibilidad.

Esto también cambia qué significa una ventana de mantenimiento. Hoy tocar la base es tocar la recepción; en el diseño propuesto, son dos cosas distintas.

Está en todos lados menos en un lugar.

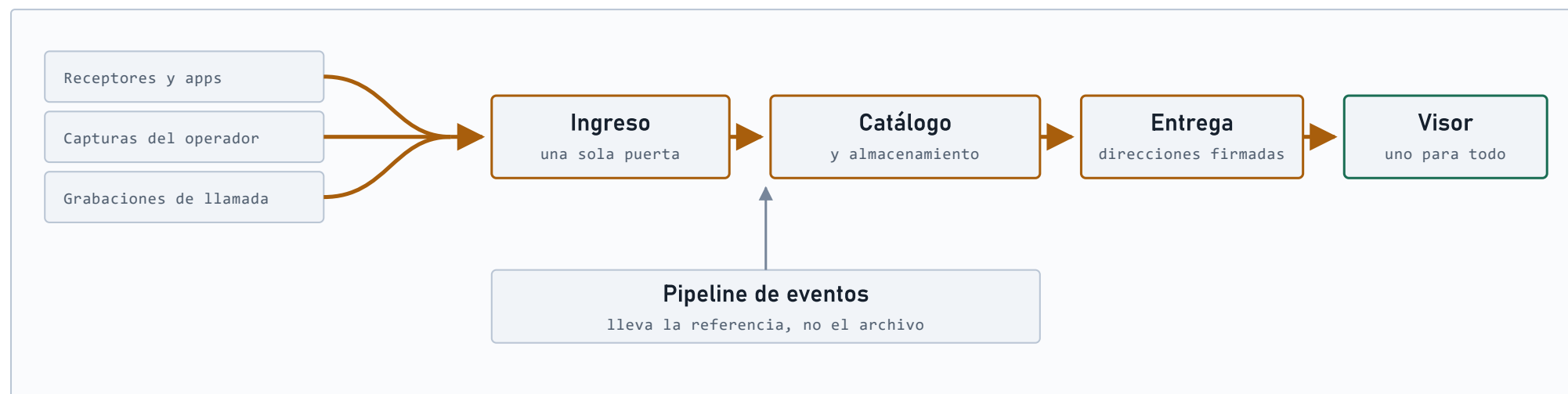
Un archivo llega con un evento y se dispersa: va a una carpeta cuyo nombre es una convención de fecha y cuenta, se anota en tablas distintas según sea imagen, video o audio, y termina en un punto de lectura propio. El navegador arma la dirección para pedirlo **concatenando la estructura de carpetas del servidor**.



Y ADEMÁS Cuando se depura un evento se borra su registro, pero **el archivo queda en el disco**. El almacenamiento sólo crece, y el sistema deja de saber que ese contenido existe.

Un solo lugar por donde la multimedia entra y sale.

Un servicio de medios se hace cargo de recibir, guardar, convertir, entregar y reproducir. Quien produce un archivo no elige carpeta; quien lo consume no arma direcciones. Y el pipeline de eventos **transporta una referencia, nunca el archivo.**



ALCANCE Este corte cubre el contenido **grabado**: imágenes, video y audio. Ver una cámara en vivo queda afuera, porque es un problema de conectividad hacia la red del cliente y no de formato: se resuelve por separado.

Un catálogo, direcciones y una política de retención.

El mismo criterio que aplicamos a los eventos: cada componente hace una cosa, y el cliente deja de conocer cómo está guardado lo que pide.

Un catálogo, no una tabla por tipo

Una sola pregunta responde qué archivos tiene un evento o una cuenta, sin importar si son imágenes, video o audio.

El cliente deja de conocer el disco

Recibe direcciones listas para usar, con vencimiento. Mover el almacenamiento deja de romper las pantallas.

Almacenamiento intercambiable

Disco local en la instalación del cliente o almacenamiento en la nube, con la misma interfaz. Es el mismo argumento de portabilidad que la base.

La retención borra de verdad

Una política declarada elimina el registro y el archivo juntos, en lugar de dejar contenido acumulándose sin que nadie lo sepa.

Las conversiones de formato también dejan de ser una cola sin reintentos: pasan a ser trabajos con reintento y cuarentena, y un archivo que falló se ve como fallado en lugar de aparecer roto en la pantalla.

No es una reescritura.

La propuesta mueve responsabilidades de lugar, una por vez. Buena parte de lo que hoy funciona se queda donde está — y eso es deliberado, porque es lo que permite avanzar sin apostar todo de una vez.

SE CONSERVA

IPRS y sus parsers

El panel se sigue conectando a IPRS igual que hoy, con los mismos protocolos. Cambia a dónde envía IPRS lo que recibe, no cómo lo recibe.

SE CONSERVA

La instalación local

El mismo producto corre en servidores del cliente o en la nube. No obliga a migrar a un modelo de servicio.

SE CONSERVA

La operación durante la transición

Los dos caminos conviven. Se habilita el nuevo para un subconjunto acotado y se vuelve atrás si algo no cierra.

SE CONSERVA

Las pantallas actuales

La primera etapa elimina la consulta en ciclo sin rehacer la interfaz. El operador ve lo mismo, actualizado antes.

SE CONSERVA

La multimedia ya grabada

Los archivos existentes se siguen sirviendo desde donde están: el catálogo aprende a encontrarlos antes de mover nada.

No hay una fecha de encendido. Hay un método que se repite.

No se arranca con un recorrido completo de punta a punta. Cada módulo se reescribe por separado con el mismo ciclo, y **cada uno entrega valor por sí solo**: no hay que esperar a que termine todo para ver resultados.



LO QUE CADA MÓDULO DEMUESTRA ANTES DE AMPLIARSE

EQUIVALENCIA Mismo resultado que el camino actual, sobre tráfico real.

RECUPERACIÓN Se mata cualquier proceso y se recupera sin perder ni duplicar.

REVERSIBILIDAD Se vuelve atrás sin migrar datos de regreso.

POR QUÉ ASÍ

Un encendido único obliga a sostener dos sistemas completos sin poder comparar equivalencia hasta el final, que es exactamente cuando ya no se puede volver atrás.

Reescribir de a un módulo mantiene el riesgo acotado a la pieza en curso, y convierte cada paso en algo que se puede medir, defender o